



## 第四章

# 堆和不相交集数据结构

朱容波

Email: [rongbozhu@126.com](mailto:rongbozhu@126.com)

Office: 一综B323

华中农业大学信息学院



# 本章内容

- 堆
  - 堆上的运算(sift-up, sift-down, insert, delete)
  - 创建堆
  - 堆排序
  - 最小堆和最大堆
- 不相交集数据结构
  - Representation
  - Union-find 算法
  - union-find 算法的分析

华中农业大学信息学院



## 4.2 堆



为什么需要堆?



在很多算法中，需要支持下面两种运算的数据结构：**插入元素和寻找最大元素**。

- 使用**普通队列**，寻找最大元素需要搜索整个队列，开销比较大。
- 采用**排序数组**，插入运算需要移动很多元素，开销也会比较大。
- 优先队列使用堆作为数据结构，更加有效的支持插入和寻找最大值元素这两种运算，

华中农业大学信息学院



## 4.2 堆 - cont'd



### 什么是堆?



#### 定义 4.1

一个（二叉）堆是一个几乎完全的二叉树，它的每个节点都满足堆的特性：如果 $v$ 和 $p(v)$ 分别是节点和它的父节点，那么存储在 $p(v)$ 中的数据项键值不小于存储在 $v$ 中数据项的键值。



# ? 下面哪些是堆?

1. 空树

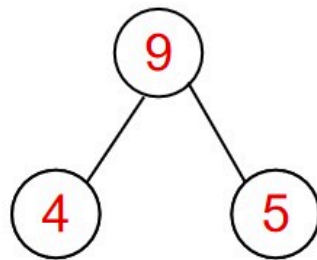
( ☒ )

2. 单节点树

(20)

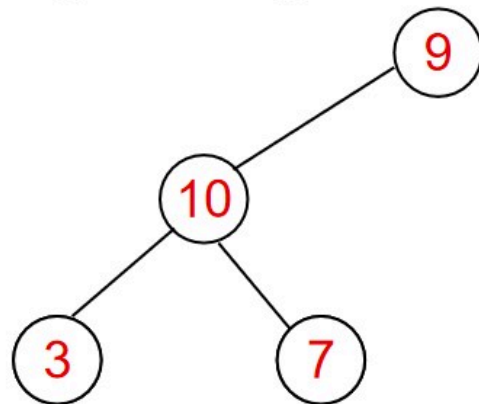
( ☒ )

3.



( ☒ )

4.

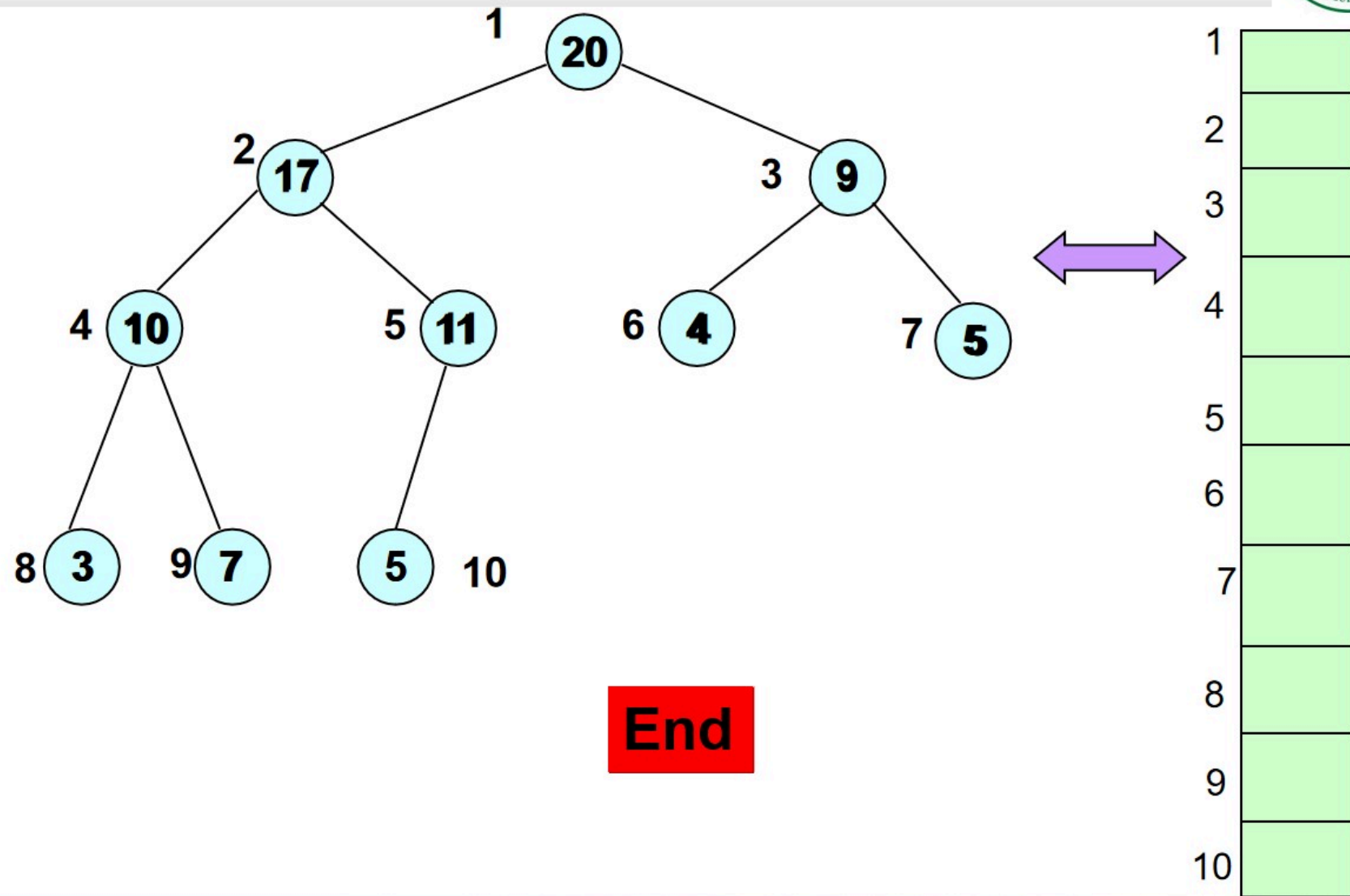


( ☒ )



## 堆的表示方式

- 一个有 $n$ 个节点的堆 $T$ ，可以由一个数组 $H[1..n]$  用下面的方式表示：
  - $T$ 的根节点存储在 $H[1]$ 中
  - 假设 $T$ 的节点 $x$ 存储在 $H[j]$ 中，如果它有左子节点，这个子节点存储在 $H[2j]$ 中；如果它也有右子节点，这个子节点存储在 $H[2j+1]$ 中
  - 元素 $H[j]$ 的父节点如果不是根节点，则存储在 $H[\lfloor j/2 \rfloor]$ 中





## 堆的特性

- 堆的特性蕴含着:沿着每条从根到叶子的路径,元素的键值以非升序排列。
- 因此堆可以看做是二叉树,而它实质上是一个数组 $H[1..n]$ 。它有如下性质:对于任何索引 $j$ ,  $2 \leq j \leq n$ ,  $\text{key}(H[\lfloor j/2 \rfloor]) \geq \text{key}(H[j])$ 。
- 如果树的节点以自顶向下、从左到右的方法,按1到 $n$ 的顺序编号,那么,每一项 $H[i]$ 在对应的树中表示成编号为 $i$ 的节点



## 堆数据结构支持的运算

- *delete-max*[H]: 从一个非空的堆H中删除最大键值的数据项并将数据项返回。
- *insert*[H,x]: 插入项x到堆H中。
- *delete*[H,i]: 从堆H中删除第i项。
- *makeheap*[A]: 将数组A转换成堆。





## 4.2.1 堆上的运算

- Sift-up
- Sift-down
- Insert
- Delete
- Delete-max



## Sift-up



什么时候使用?



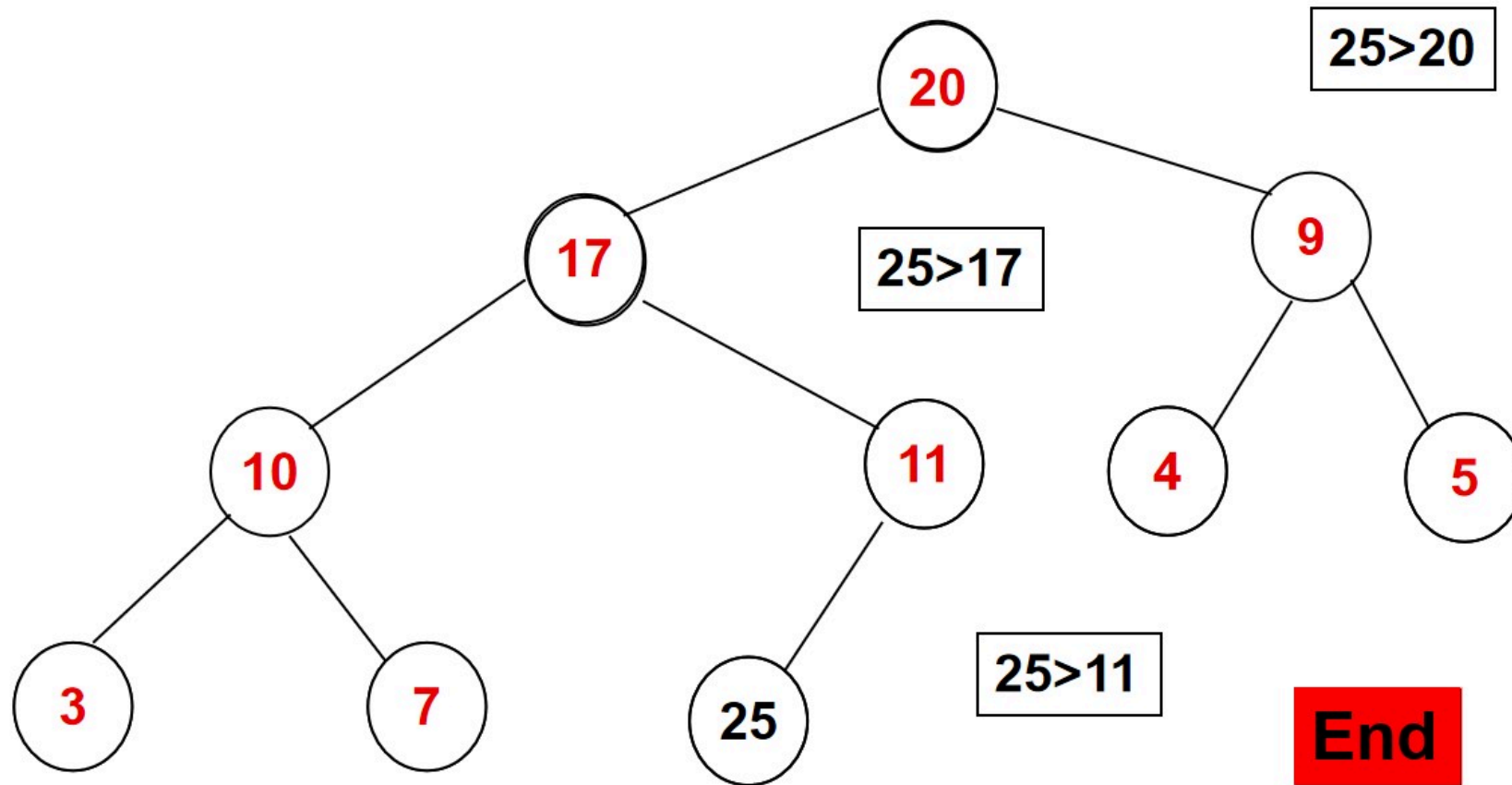
假定对于某个 $i > 1$ ,  $H[i]$ 变成了键值大于它父节点键值的元素, 这样就违反了堆的特性, 因此这种数据结构就不再是堆了。如要修复堆的特性, 需要用称为Sift-up的运算把新的数据项上移到在二叉树中适合它的位置上, 这样堆的属性就修复了。

sift-up 运算沿着从 $H[i]$ 到根节点的惟一一条路径, 把 $H[i]$ 移到适合它的位置上。

在沿着路径的每一步上, 都将 $H[i]$ 键值和它父节点的键值 $H[\lfloor i/2 \rfloor]$ 相比较。



## 例子





## 过程 SIFT-UP

输入: 数组  $H[1 \dots n]$  和位于 1 和  $n$  之间的索引  $i$

输出: 上移  $H[i]$  (如果需要), 以使它不大于父节点。

1.  $done \leftarrow false$
2. if  $i=1$  then exit                      {节点  $i$  为根}
3. repeat
4.    if  $key(H[i]) > key(H[\lfloor i/2 \rfloor])$  then 交换  $H[i]$  and  $H[\lfloor i/2 \rfloor]$
5.    else  $done \leftarrow true$
6.     $i \leftarrow \lfloor i/2 \rfloor$
7. until  $i=1$  or  $done$

## Sift-down



什么时候使用?



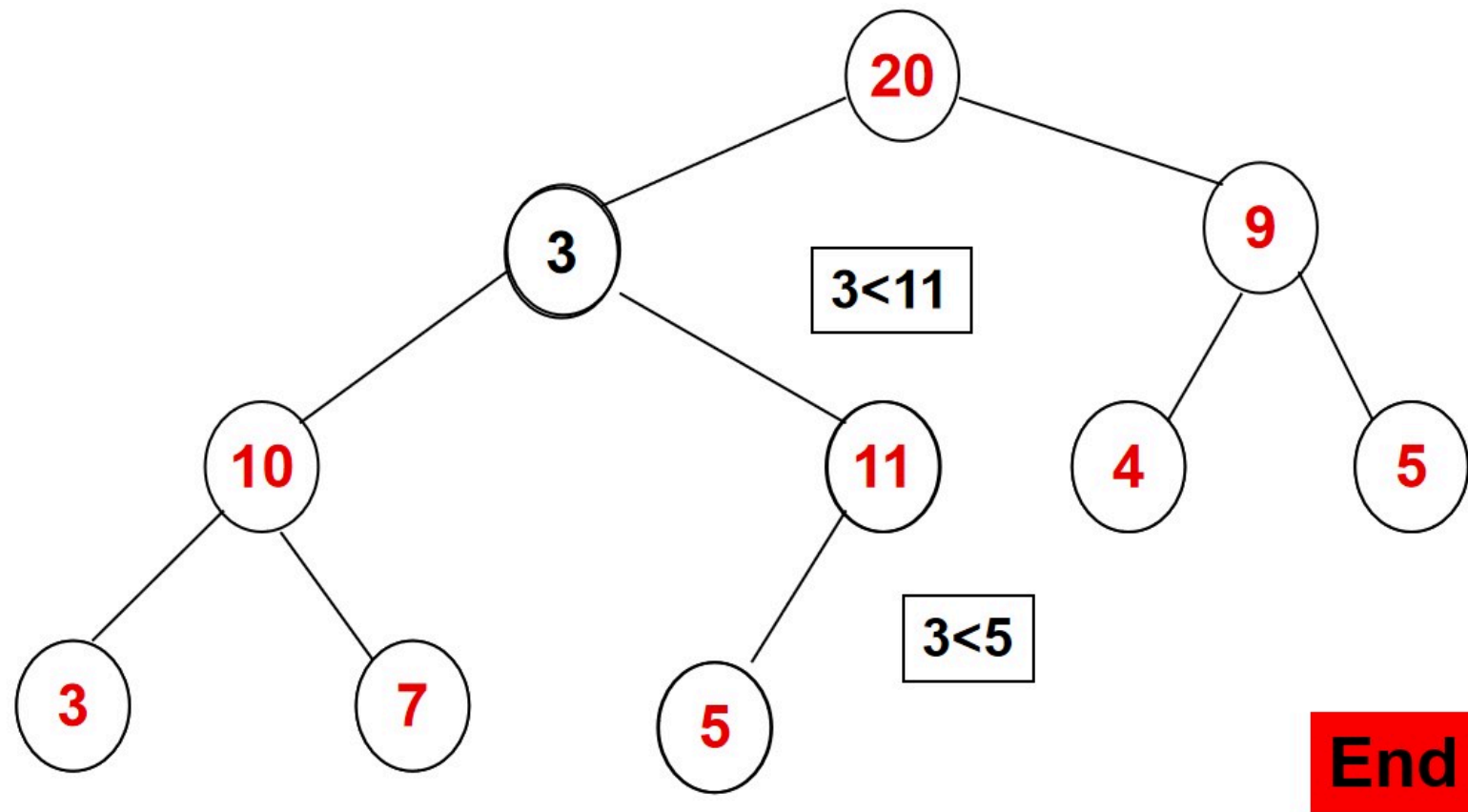
假定对于 $i \leq \lfloor n/2 \rfloor$ , 存储在 $H[i]$ 中元素的键值变成小于 $H[2i]$ 和 $H[2i+1]$ 中的最大值(如果 $H[2i+1]$ 存在的话), 这样就违反了堆的特性, 树就不再表示一个堆。

- 如要修复堆的特性, 需要用Sift-down运算使 $H[i]$ “渗”到二叉树中适合它的位置上, 沿着这条路径的每一步, 都把 $H[i]$ 的键值和存储在它子节点(如果存在)中的两个键值里最大的那个相比较。





## An example



## 过程 SIFT-DOWN

输入: 数组  $H[1...n]$  和位于 1 和  $n$  之间的索引  $i$

输出: 下移  $H[i]$  (如果需要), 以使它不小于子节点。

1.  $done \leftarrow false$
2. if  $2i > n$  then exit                      {节点  $i$  是叶子}
3. repeat
4.     $i \leftarrow 2i$
5.    if  $i+1 \leq n$  and  $key(H[i+1]) > key(H[i])$  then  $i \leftarrow i+1$
6.    if  $key(H[\lfloor i/2 \rfloor]) < key(H[i])$  then 互换  $H[i]$  and  $H[\lfloor i/2 \rfloor]$
7.    else  $done \leftarrow true$
8.    end if
9. until  $2i > n$  or  $done$



# 插入

- **基本思想:** 为了把元素 $x$ 插入到堆 $H$ 中, 先将堆大小加1, 然后将 $x$ 添加到 $H$ 的末尾, 再根据需要, 把 $x$ 上移, 直到满足堆的特性。

## 算法 4.1 INSERT

输入: 堆 $H[1...n]$  和元素 $x$ 。

输出: 新的堆  $H[1...n+1]$ ,  $x$ 为其元素之一。

1.  $n \leftarrow n+1$                       {增加 $H$ 的大小}
2.  $H[n] \leftarrow x$
3. SIFT-UP( $H, n$ )

时间复杂度:  $O(\log n)$



# 删除

- 基本思想:

- 要从大小为 $n$ 的堆 $H$ 中删除元素 $H[i]$ , 可先用 $H[n]$ 替换 $H[i]$ , 然后将堆栈大小减1, 如果需要的话, 根据 $H[i]$ 的键值与存储在它父节点和子节点中元素键值的关系, 对 $H[i]$ 做Sift-up或Sift-down运算, 直到满足堆特性为止。



## 算法 4.2 DELETE

输入:非空堆 $H[1...n]$ 和位于1和 $n$ 之间的索引 $i$ 。

输出:删除 $H[i]$ 之后的新堆 $H[1...n-1]$ 。

1.  $x \leftarrow H[i];$                        $y \leftarrow H[n]$
2.  $n \leftarrow n-1$                       {decrease the size of  $H$ }
3. **if**  $i=n+1$  **then exit**      {done}
4.  $H[i] \leftarrow y$
5. **if**  $\text{key}(y) \geq \text{key}(x)$  **then** SIFT-UP( $H, i$ )
6. **else** SIFT-DOWN( $H, i$ )
7. **end if**

时间复杂度:  $O(\log n)$ .





# 删除最大值

- **基本思想:** This 这项运算在一个非空堆H中删除并返回最大键值的数据项。直接完成这项运算要用到删除运算:只要返回根节点中的元素并将其从堆中删除。

## 算法 4.3 DELETEMAX

输入: 堆H[1...n].

输出: 返回最大键值元素x并将其从堆中删除。

1.  $x \leftarrow H[1]$
2. DELETE(H,1)
3. return x

时间复杂度:  $O(\log n)$



## 4.2.2 创建堆

- **方法1:**

- 给出一个有 $n$ 个元素的数组 $A[1..n]$ ，很容易创建一个包含这些元素的堆，可以这样进行:从空的堆开始，不断插入每一个元素，直到 $A$ 完全被转移到堆中为止。

将数组 $A[1..10]=\{4,3,8,10,11,13,7,30,17,26\}$  转换成一个堆。

- 时间复杂度:

因为插入第 $j$ 个键值用时 $O(\log j)$ ，因此用这种方法创建堆栈的时间复杂性是 $O(n \log n)$ 。



## 4.2.2创建堆- cont'd

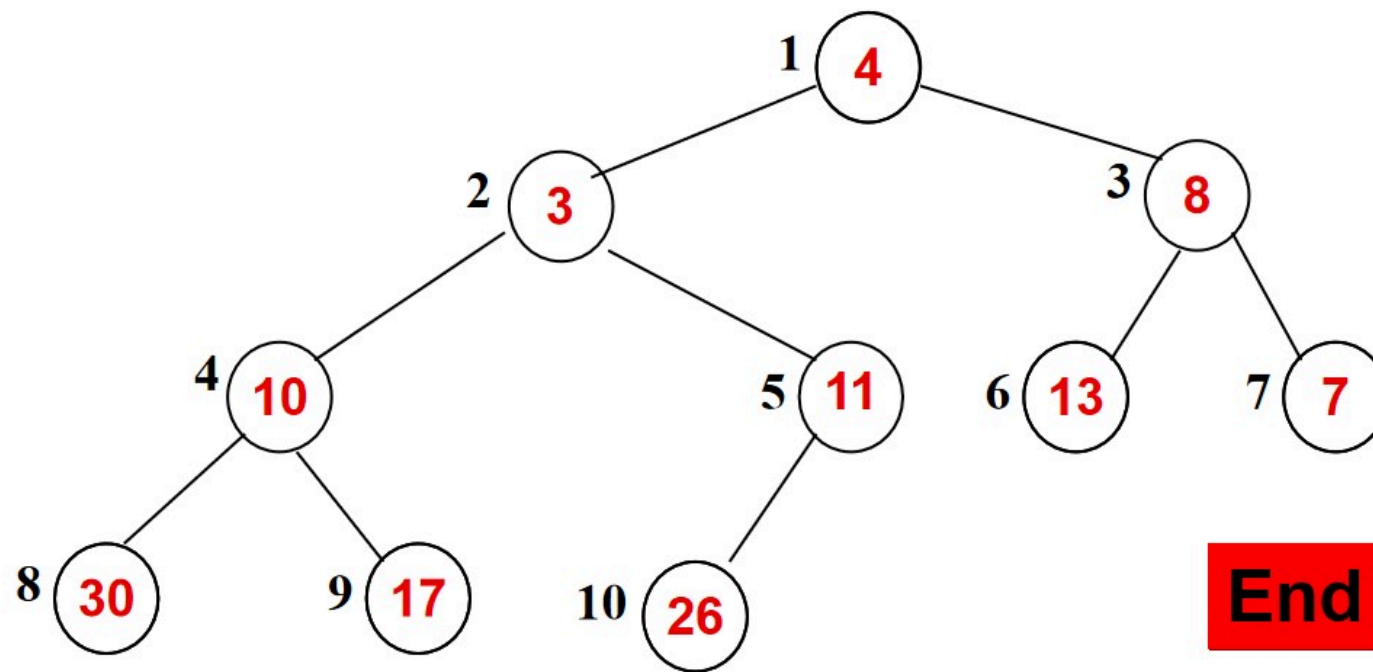
- **方法 2:**

- 把一棵n个节点的几乎完全的二叉树转换成堆 $H[1..n]$ 。  
从最后一个节点开始(编码为n的那一个)到根节点(编码为1的节点), 逐个扫描所有的节点, 根据需要, 每一次将以当前节点为根节点的子树转换成堆。
- 这些元素 $A[\lfloor n/2 \rfloor + 1], \dots, A[n]$ 它们对应于T的叶子, 这样可以从 $A[\lfloor n/2 \rfloor]$ 开始调整数组, 并且继续 $A[\lfloor n/2 \rfloor - 1], \dots, A[1]$ 进行调整。



## 例子

将数组 $A[1..10]=\{4,3,8,10,11,13,7,30,17,26\}$  转换成一个堆.







## 算法 4.4 MAKEHEAP

输入:  $n$  个元素的数组  $A[1..n]$  。

输出:  $A[1..n]$  转换成堆

1. **for**  $i \leftarrow \lfloor n/2 \rfloor$  **downto** 1
2.     SIFT-DOWN( $A, i$ )
3. **end for**





# MAKEHEAP算法分析

- 设T是对应于数组A[1..n]的一棵几乎完全的二叉树，那么由观察结论3.4可知，T的高是 $k = \lfloor \log n \rfloor$ 。
- 令A[j]对应该树的第i层中第j个节点，当语句SIFT-DOWN(A, j)，调用过程SIFT-DOWN时，重复执行的次数最多是k-i
- 因为在第i层上正好有 $2^i$ 个节点， $0 \leq i < k$ ，循环执行的总次数的上界是：

$$\begin{aligned} \sum_{i=0}^{k-1} (k-i)2^i &= 2^0 \cdot k + 2^1 \cdot k^2 + \dots + 2^{k-1} \cdot 1 = \sum_{i=1}^k i \cdot 2^{k-i} = 2^k \sum_{i=1}^k i/2^i \\ &\leq n \sum_{i=1}^k i/2^i < 2n \quad (\text{see equation 2.14, p78(49)}) \end{aligned}$$

华中农业大学信息学院

# MAKEHEAP算法分析

- 由于在过程SIFT-DOWN的每一个循环中，最多有两次元素的比较。
- 因此元素比较的总次数的上界是 $4n$ .
- 而且因为在每次调用过程SIFT-DOWN时，都要至少执行一次循环，元素比较的最小次数是 $2\lfloor n/2 \rfloor \geq n-1$ .



- **定理 4.1**

算法MAKEHEAP用来构造一个 $n$ 元素的堆，令 $C(n)$ 为执行该算法的元素比较次数，那么 $n-1 \leq C(n) < 4n$ 。

- 时间复杂度:  $\Theta(n)$ , 空间复杂度:  $\Theta(1)$



## 4.2.3 堆排序

### 基本思想

给出数组 $A[1...n]$ ,

- 首先将A变换成堆, 并使其具有这样的性质, 每个元素的键值是该元素本身, 即 $\text{key}(A[i])=A[i]$ 。
- 由于A中各项的最大值存储在 $A[1]$ 中, 可以将 $A[1]$ 和 $A[n]$ 交换, 使得 $A[n]$ 是数组中最大元素。这时,  $A[1]$ 中的元素可能小于存放在它的一个子节点中的元素, 于是用过程SIFT-DOWN将 $A[1...n-1]$ 转换成堆。
- 接下来将 $A[1]$ 和 $A[n-1]$ 交换, 并调整数组 $A[1...n-2]$ 成为堆。
- 交换元素和调整堆的过程一直重复, 直到堆的大小变成 $i$ 为止, 这时,  $A[1]$ 是最小的。

华中农业大学信息学院





## 算法 4.5 HEAPSORT

输入:  $n$  个元素的数组  $A[1 \dots n]$  。

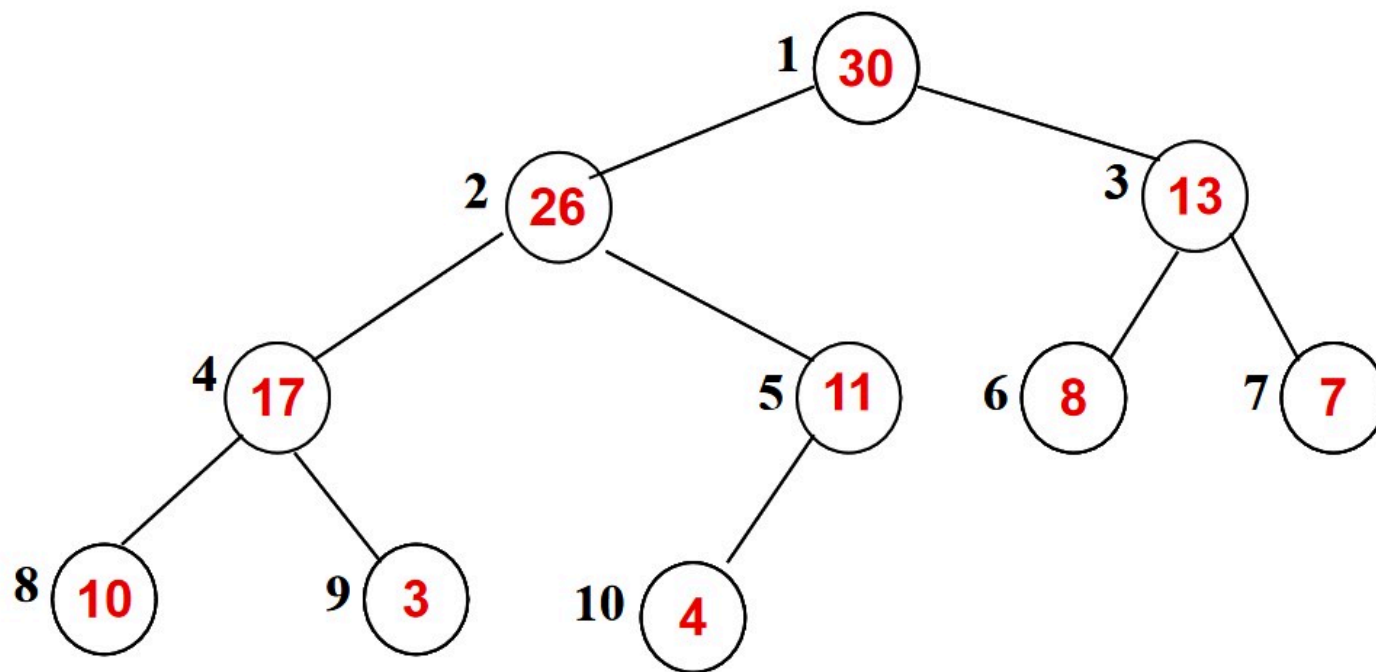
输出: 以非降序排列的数组。

1. MAKEHEAP( $A$ )
2. for  $j \leftarrow n$  downto 2
3.     互换  $A[1]$  and  $A[j]$
4.     SIFT-DOWN( $A[1 \dots j-1], 1$ )
5. end for



## 例子

- 将数组  $A[1..10] = \{4, 3, 8, 10, 11, 13, 7, 30, 17, 26\}$  进行非降排序



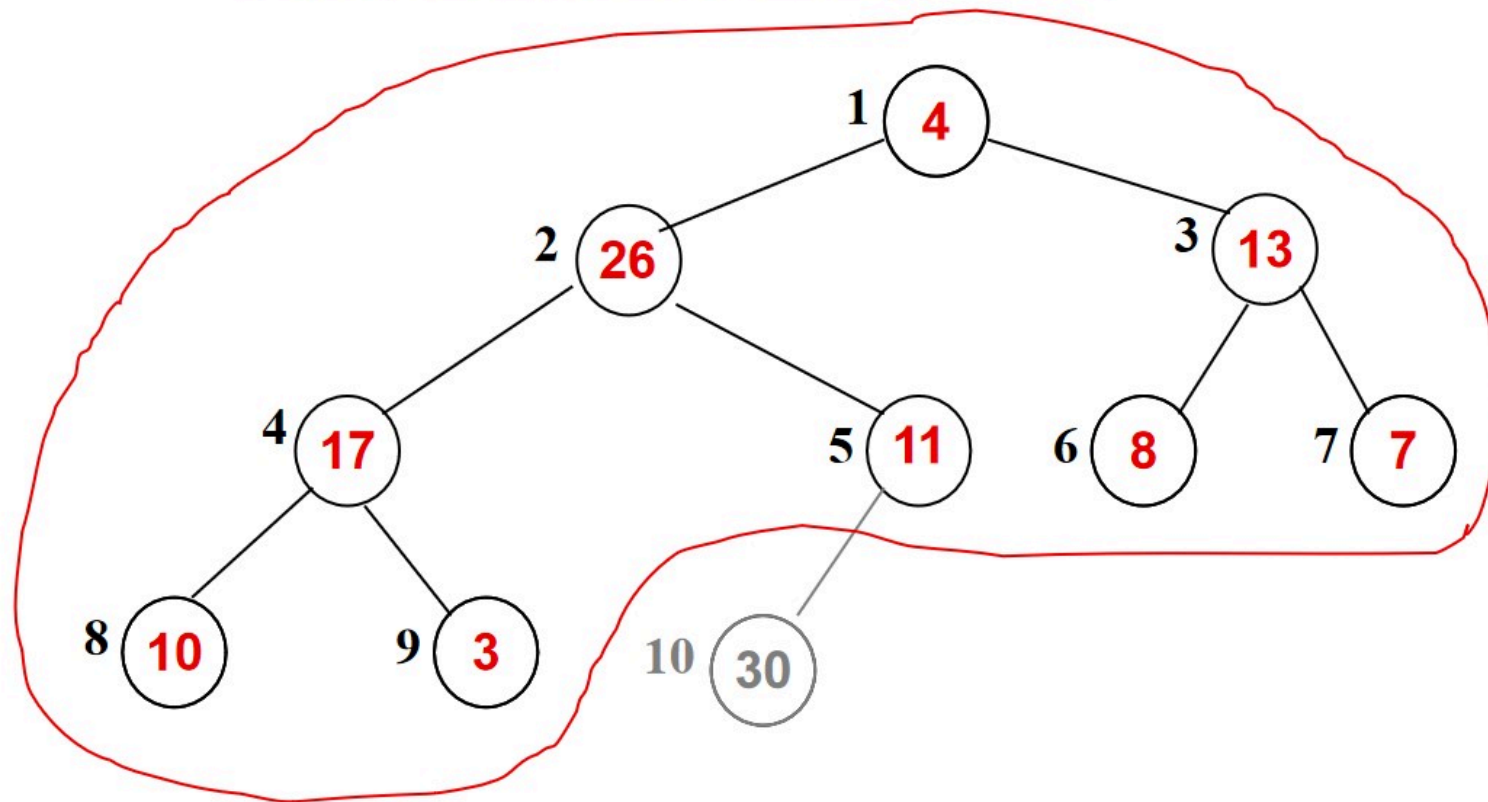
|    |    |
|----|----|
| 1  | 30 |
| 2  | 26 |
| 3  | 13 |
| 4  | 17 |
| 5  | 11 |
| 6  | 8  |
| 7  | 7  |
| 8  | 10 |
| 9  | 3  |
| 10 | 4  |





## An example

- Sort the array  $A[1..10]=\{4,3,8,10,11,13,7,30,17,26\}$  in non-descending order.

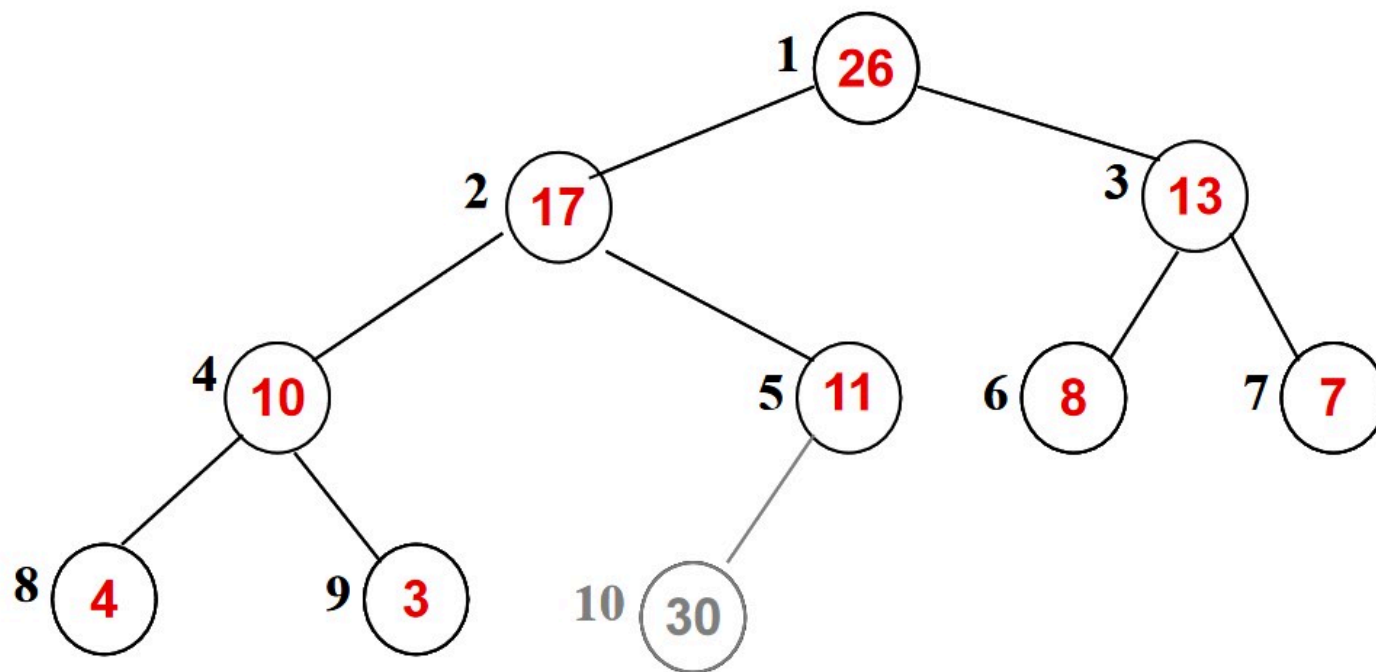


|    |    |
|----|----|
| 1  | 4  |
| 2  | 26 |
| 3  | 13 |
| 4  | 17 |
| 5  | 11 |
| 6  | 8  |
| 7  | 7  |
| 8  | 10 |
| 9  | 3  |
| 10 | 30 |



## An example

- Sort the array  $A[1..10]=\{4,3,8,10,11,13,7,30,17,26\}$  in non-descending order.



|    |    |
|----|----|
| 1  | 26 |
| 2  | 17 |
| 3  | 13 |
| 4  | 10 |
| 5  | 11 |
| 6  | 8  |
| 7  | 7  |
| 8  | 4  |
| 9  | 3  |
| 10 | 30 |



# 算法分析

- 空间：
  - 这个算法的一个重大好处在于它是在原有的空间里排序，它不需要辅助存储空间。
- 时间：
  - 建立堆用 $\Theta(n)$ 时间
  - Sift-down运算用 $O(\log j)$ 时间，并且要重复 $n-1$ 次，



## Theorem 4.2

算法HEAPSORT对 $n$ 个元素排序要用 $\Theta(n \log n)$ 时间和 $\Theta(1)$ 空间。



## 4.2.4 最小堆和最大堆

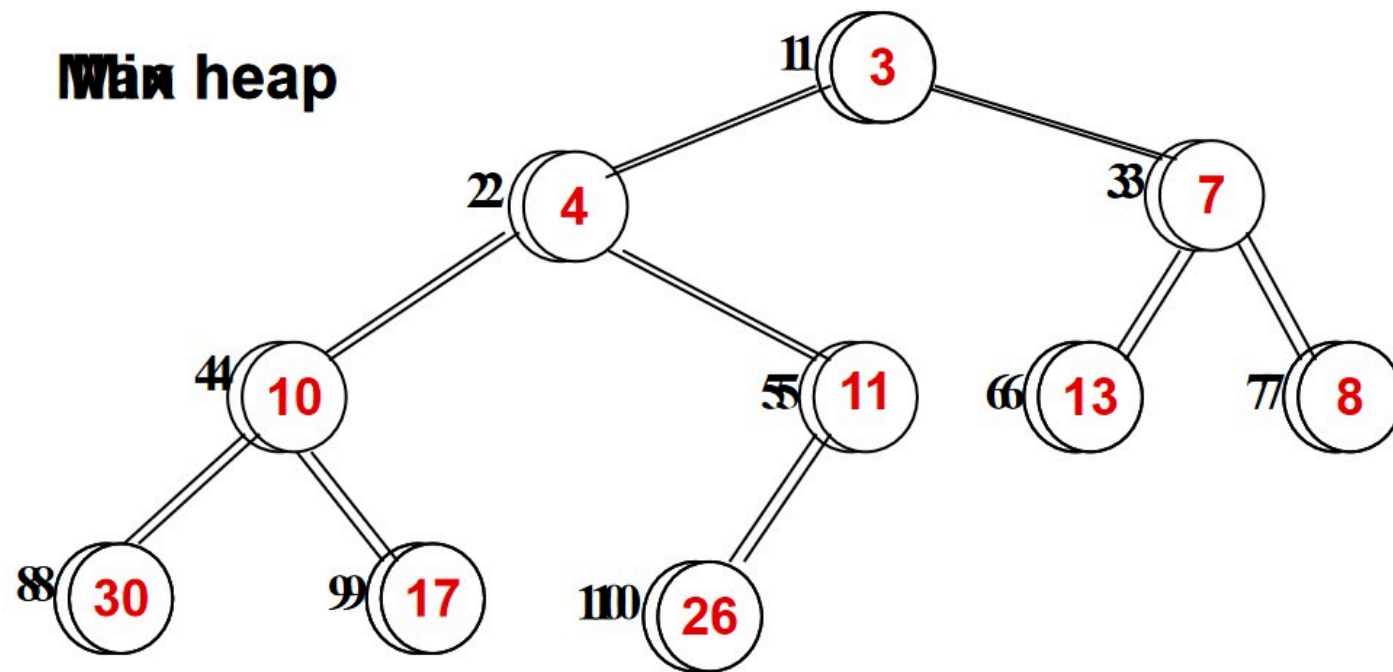
- 最大堆:
  - 存储在根节点以外的节点的键值, 小于或等于存储在它父节点中的键值。
- 最小堆:
  - 存储在根节点以外的节点的键值, 大于或等于存储在它父节点中的键值。





## 例子

$A[1..10] = \{4, 3, 8, 10, 11, 13, 7, 30, 17, 26\}$  转换成堆.





# 堆应用-最小耗费生成树(Prim算法)

 定义:

- 设 $G = (V, E)$ 是一个具有含权边的连通无向图。 $G$ 的一棵生成树 $(V, T)$ 是 $G$ 的作为树的子图。如给 $G$ 加权并且 $T$ 的各边的权的和为最小值, 那么 $(V, T)$ 就称为**最小耗费生成树**或简称为**最小生成树**

## ? 问题描述

- 假设 $G$ 为连通的, 怎么找到最小生成树 (MST)?

华中农业大学信息学院



# 数据结构设计在算法中的作用

## - 最小耗费生成树 (Prim算法)

- 基本思想
- 设 $G=(V,E)$ , 为了简便起见,  $V$ 取整数集合  $\{1,2,\dots,n\}$ .
- 算法从建立两个顶点集合开始:  $X=\{1\}$  和  $Y=\{2,\dots,n\}$ 。接着生长一棵生成树, 每次一条边。
- 在每一步, 它找出一条权重最小的边 $(x,y)$ , 这里 $x \in X$ ,  $y \in Y$ 并且把 $y$ 从 $Y$ 移到 $X$ 。重复这一步直到 $Y$ 为空。

华中农业大学信息学院



# Prim算法

此算法概况如下:

1.  $T \leftarrow \{\}$ ;  $X \leftarrow \{1\}$ ;  $Y \leftarrow V - \{1\}$

2. **while**  $Y \neq \{\}$

3. 设 $(x,y)$ 是最小权重的边, 其中 $x \in X$  and  $y \in Y$

4.  $X \leftarrow X \cup \{y\}$

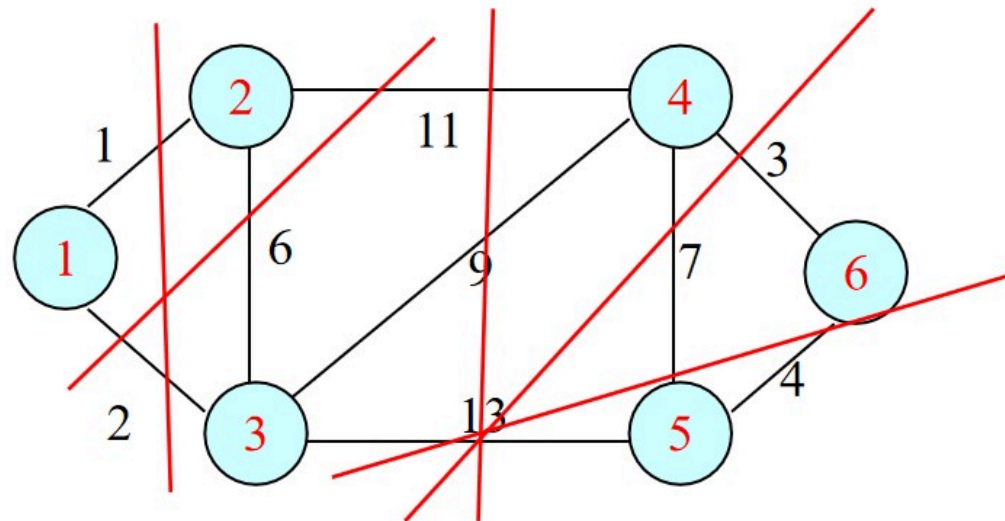
5.  $Y \leftarrow Y - \{y\}$

6.  $T \leftarrow T \cup \{(x,y)\}$

7. **end while**



# 例子



Success!



## 算法 8.4 PRIM p154

输入:含权连通无向图 $G=(V,E)$ , 其中  $V=\{1,2,\dots,n\}$ .

输出:由 $G$ 生成的最小耗费生成树 $T$ 组成的边的集合。

```
1.  $T \leftarrow \{\}$ ;  $X \leftarrow \{1\}$ ;  $Y \leftarrow V - \{1\}$ 
2. for  $y \leftarrow 2$  to  $n$ 
3.     if  $y$  邻接于 1 then
4.          $N[y] \leftarrow 1$ 
5.          $C[y] \leftarrow c[1,y]$ 
6.     else  $C[y] \leftarrow \infty$ 
7.     end if
8. end for
```

NEXT PAGE



## 算法 8.4 PRIM

```
9. for j←1 to n-1 {寻找n-1 条边}
10.     令y∈Y 使得 C[y] 最小
11.     T←T∪{y,N[y]}           {将边(y,N[y]) 加入 T}
12.     X←X∪{y}                 {将顶点y 加入 X}
13.     Y←Y-{y}                 {从Y删除顶点}
14.     for 每个邻接于y的顶点 w ∈ Y
15.         if c[y,w]<C[w] then
16.             N[w]←y
17.             C[w]←c[y,w]
18.         end if
19.     end for
20. end for
```



# 算法分析

- 第1步耗费  $\Theta(n)$  时间.
- 第2步的for循环需要时间  $\Theta(n)$ . 第3步到第6步需要时间  $O(n)$ .
- 第10步搜索离X最近的顶点y, 每迭代一次需要花费时间  $\Theta(n)$ , 而这一步执行了  $n-1$  次, 所以第10步一共花费时间  $\Theta(n^2)$
- 第11步、第12步和第13步, 每迭代一次花费时间  $\Theta(1)$  总共花费时间  $\Theta(n)$ 。
- 第14步的for循环执行了  $2m$  次, 第14步一共需要的时间是  $\Theta(m)$
- ...P156
- 这就得到了算法的时间复杂性是  $\Theta(m+n^2)$

华中农业大学信息学院





# 堆在算法中的作用- 稠图的线性时间算法

## 基本思想

- 现在要改进算法PRIM，就像曾经对算法DIJKSTRA所做的那样，目标是把 $m = o(n^2)$ 的那类图的时间复杂性从 $\Theta(n^2)$ 减少到 $O(m \log n)$ 。以后还要进一步改进它，使在稠图情况下，可以在对边数的线性时间内运行。
- 就像在算法SHORTESTPATH中那样，基本思想是用**最小堆**数据结构(见4.2节)来保持边界顶点集，使得可以在 $O(\log n)$ 时间内取得Y集中的顶点y，这个y和V-Y集中一个顶点连接的边的耗费是最小的。修改后的算法在算法MST中给出。

华中农业大学信息学院



## 算法 3.4 MST

输入:含权连通无向图 $G=(V,E)$ , where  $V=\{1,2,\dots,n\}$ .

输出:由 $G$ 生成的最小耗费生成树 $T$ 组成的边的集合。假设我们已有一个空堆 $H$ .

```
1.       $T \leftarrow \{\}$ ;  $Y \leftarrow V - \{1\}$ 
2.      for  $y \leftarrow 2$  to  $n$ 
3.  if  $y$  邻接于 1 then
4.           $N[y] \leftarrow 1$ 
5.           $key(y) \leftarrow c[1,y]$ 
6.          INSERT( $H,y$ )
7.  else  $key(y) \leftarrow \infty$ 
8.      end if
9. end for
```

NEXT PAGE

华中农业大学信息学院



### 算法3.4 MST

```
10. for  $j \leftarrow 1$  to  $n-1$                                 {查找 $n-1$  条边}
11.    $y \leftarrow \text{DELETEMIN}(H)$ 
12.    $T \leftarrow T \cup \{(y, N[y])\}$                         {添加边 $(y, N[y])$ 到 $T$ }
13.    $Y \leftarrow Y - \{y\}$                                 {从 $Y$ 删除顶点 $y$ }
14.   for 每个邻接于 $y$ 的顶点 $w \in Y$ 
15.     if  $c[y, w] < \text{key}(w)$  then
16.        $N[w] \leftarrow y$ 
17.        $\text{key}(w) \leftarrow c[y, w]$ 
18.     end if
19.     if  $w \notin H$  then  $\text{INSERT}(H, w)$ 
20.     else  $\text{SIFTUP}(H, H^{-1}(w))$ 
21.   end for
22. end for
```

华中农业大学信息学院



# 算法分析

## 定理3.4

- 给出一个含权无向图 $G$ ，算法MST在 $O(m \log n)$ 时间内找出最小耗费生成树。如果图是稠密的，即如果对于某个 $\epsilon > 0$ ,  $m \geq n^{1+\epsilon}$ ，那么它可以被进一步改进为在 $O(m/\epsilon)$  时间内运行。



## 4.3 不相交集数据结构



什么是不相交集?



设给出一个有 $n$ 个不同元素的集合 $S$ ，这些元素被分成不相交集。在每个子集中，用一个特殊的元素作为集合的名字或代表。



例如，如果集合 $S=\{1,2,\dots,11\}$ 有4个子集，分别是 $\{1,7,10,11\}$ ,  $\{2,3,5,6\}$ ,  $\{4,8\}$  和  $\{9\}$ ，这些子集可以依次被标记为1,3,8,9。



# 不相交集的操作

- **FIND(x)**: 寻找并返回包含元素x的集合的名字。
- **UNION(x,y)**: 包含元素x和y的两个集合用它们的并集替换。并集的名字或者是原来包含元素x的那个集合的名字，或者是原来包含元素y的那个集合的名字，这将会在以后确定。



— 例如，如果集合 $S=\{1,2,\dots,11\}$ 有4个子集  $\{1,7,10,11\}$ ,  $\{2,3,5,6\}$ ,  $\{4,8\}$  和  $\{9\}$ , 这些子集可以依次被标记为1,3,8,9。

**FIND(11)** 运算返回结果 1;

**UNION(1,3)** 运算返回新子集被标记为1 或 3



# ? 我们的目的是什么?

- 我们的目的是设计有效算法进行FIND和UNION操作。
- 为此需要一种数据结构，它既要简单，同时又要考虑到能有效地实现合并和寻找这两种运算。
- How to represent a set?

# Tree 树表示法

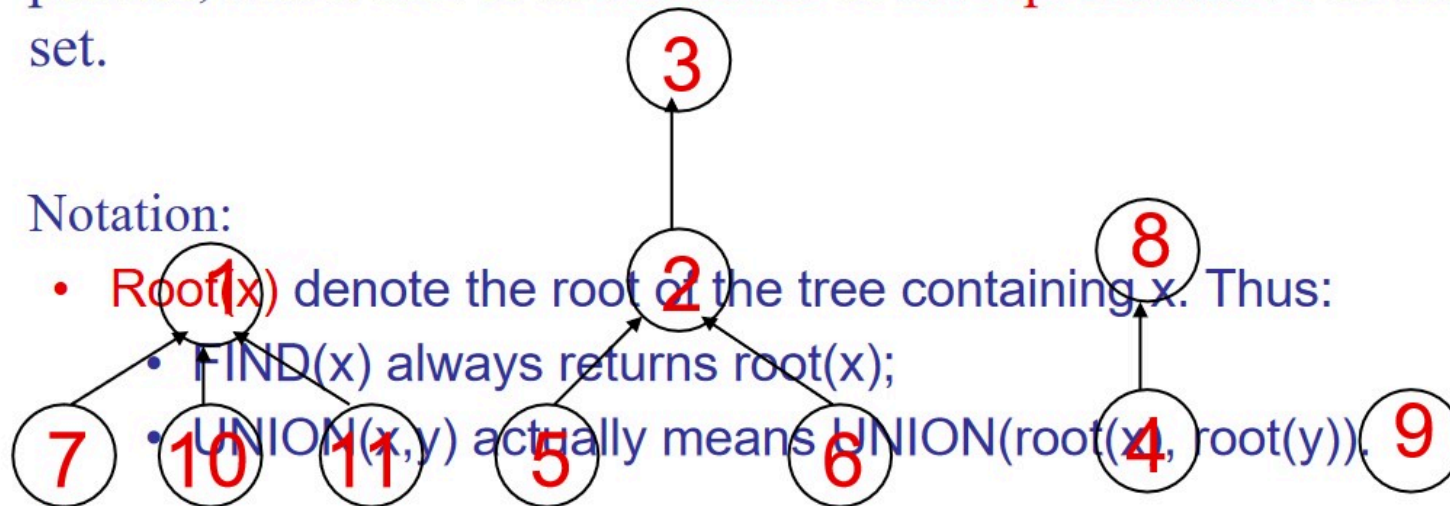
- To represent each set as a rooted tree with data elements stored in its nodes. Each element  $x$  other than the root has a pointer to its parent  $p(x)$  in the tree. The root has a null pointer, and it serves as **the name** or **set representative** of the set.

- Notation:

- Root(x)** denote the root of the tree containing  $x$ . Thus:

- FIND(x)** always returns  $\text{root}(x)$ ;

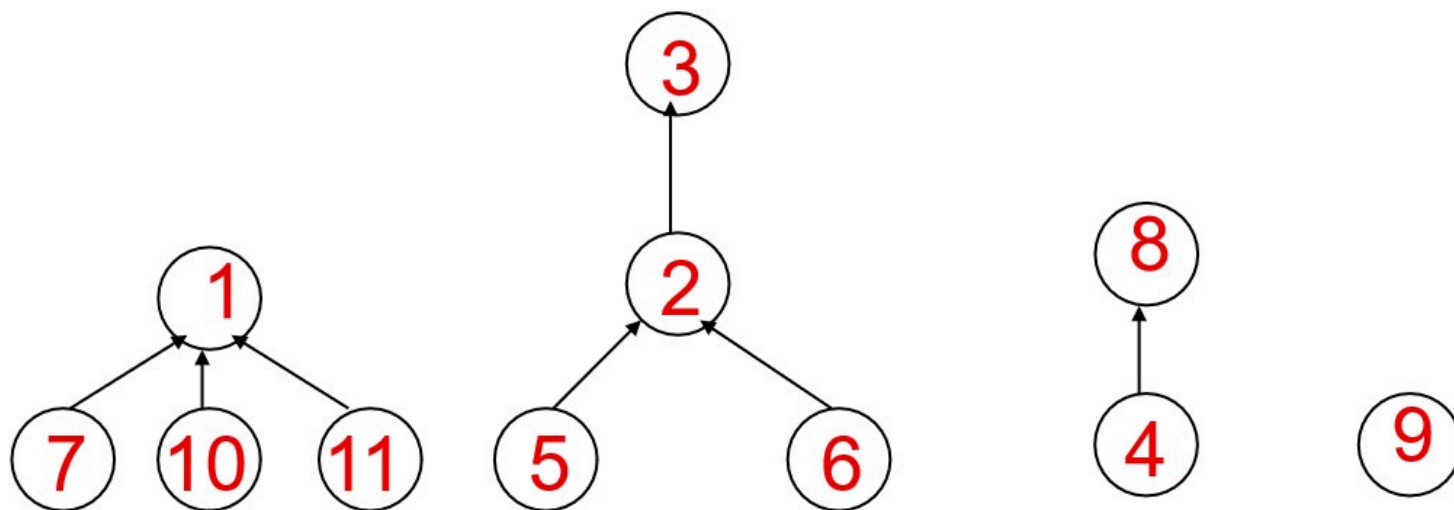
- UNION(x,y)** actually means **UNION( $\text{root}(x)$ ,  $\text{root}(y)$ )**.





## 树的表示法

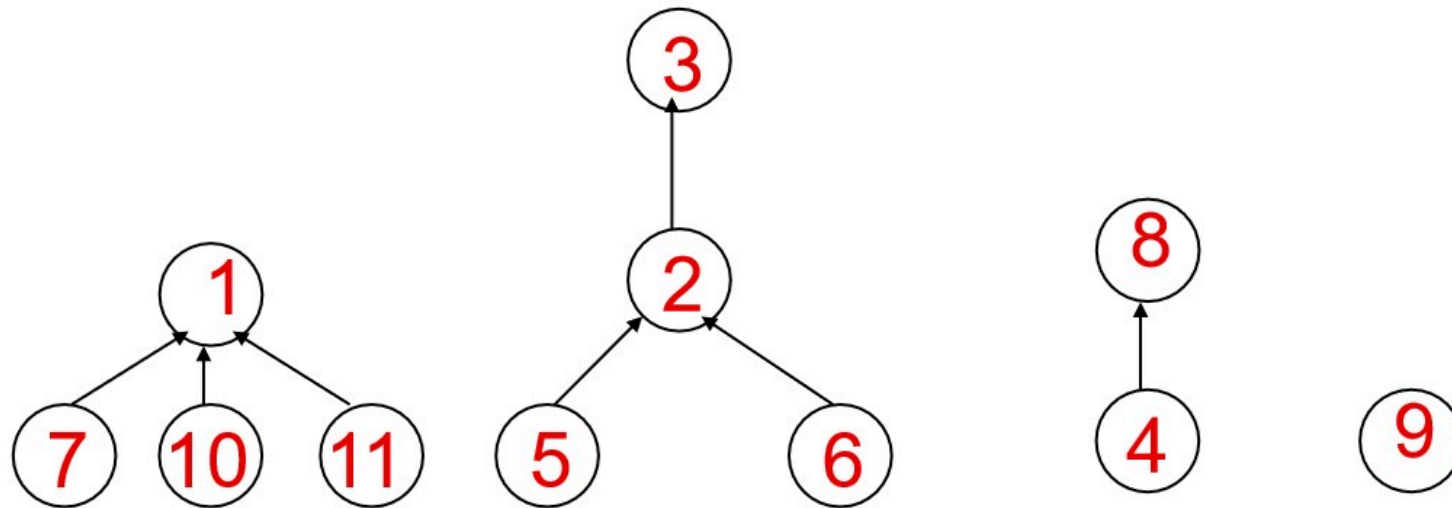
- 一个子集用一棵树表示，则所有的不相交子集组成森林。



- 如果假定元素是整数 $1, 2, \dots, n$ , 则森林可以方便地用数组  $A[1..n]$  来表示,  $A[j]$  是元素  $j$  的父节点。空的父节点可以用数字 0 来表示。



## 表示法的例子



| 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9 | 10 | 11 |
|---|---|---|---|---|---|---|---|---|----|----|
| 0 | 3 | 0 | 8 | 2 | 2 | 1 | 0 | 0 | 1  | 1  |

End



## 直接实现的方法

- **FIND(x):**

- 在进行FIND( x )运算时，只是沿着从x开始直到根节点的路径，然后返回root(x)。

- **UNION(x,y):**

- 在进行UNION 运算时，令root(x)的链接指向root(y)，也就是说，如果root(x)是u，root(y)是v，就令v是u的父节点。



### FIND(X)流程

**Input:** A node  $x$ .

**Output:**  $\text{root}(x)$ , the root of the tree containing  $x$ .

1.  $y \leftarrow x$
2. **while**  $p(y) \neq \text{null}$       {Find the root of the tree containing  $x$ }
3.      $y \leftarrow p(y)$
4. **end while**
5. **return**  $y$ ;

### UNION(x,y)流程

**Input:** Two elements  $x$  and  $y$ .

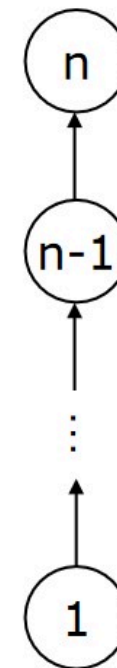
**Output:** The union of the two trees containing  $x$  and  $y$ . The original trees are destroyed

1.  $u \leftarrow \text{FIND}(x)$ ;  $v \leftarrow \text{FIND}(y)$
2.  $p(u) \leftarrow v$ ;



## Are they useful?

- 考虑以下不相交集合  $\{1\}, \{2\}, \dots, \{n\}$  以及操作  $\text{UNION}(1,2), \text{FIND}(1), \text{UNION}(2,3), \text{FIND}(1), \text{UNION}(3,4), \text{FIND}(1), \text{UNION}(4,5), \text{FIND}(1), \dots, \text{UNION}(n-1,n), \text{FIND}(1)$ .
- 得到结果如右所示:



The last *find* operation requires  $n$  times.

**?** 我们是否能重设一棵树的高度?



## 4.3.1 按秩合并措施

- 为限制每棵树的高度，采用按秩合并措施；
- 是使包含较少结点的树的根指向包含较多结点的树的根，而这个树的大小可以抽象为树的高度，即高度小的树合并到高度大的树，这样资源利用更加合理。



### 基本思想

- 给每个节点存储一个非负数作为该节点的秩，记为 $\text{rank}$ ，节点的秩基本上就是它的高度。
- 设 $x$ 和 $y$ 是当前森林中两棵不同的树的根
  - 初始状态时，每个节点的秩是0
  - 在执行运算 $\text{UNION}(x, y)$ 时，比较 $\text{rank}(x)$  和  $\text{rank}(y)$ 。
    - 如果 $\text{rank}(x) < \text{rank}(y)$ ，就使得 $y$  为 $x$  的父节点
    - 如果 $\text{rank}(x) > \text{rank}(y)$ ，就使得 $x$  为 $y$  的父节点
    - 否则，如果 $\text{rank}(x) = \text{rank}(y)$ ，就使得 $y$  为 $x$  的父节点，并将  $\text{rank}(y)$  加1。



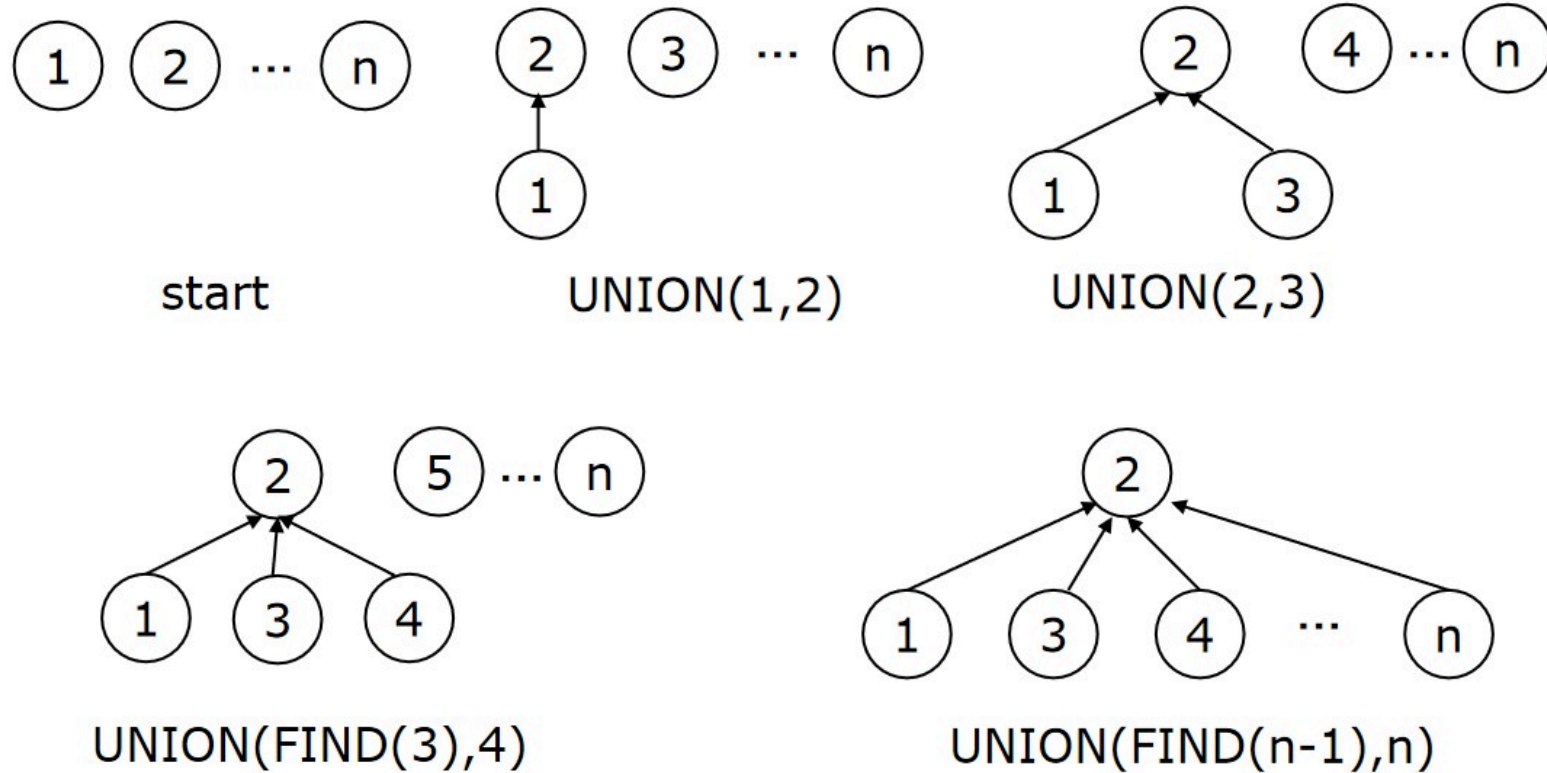
### 4.3.1 按秩合并措施

- 为了实现按秩合并不相交集森林，要记录下秩的变化。
- 对于每个结点 $x$ ，有一个整数 $\text{rank}[x]$ ，它是 $x$ 的高度（从 $x$ 到其某一个后代叶结点的最长路径上边的数目）的一个上界。（即树高）。
- 当由MAKE-SET创建了一个单元集时，对应的树中结点的初始秩为0，每个FIND-SET操作不改变任何秩。
- 当对两棵树应用UNION时，有两种情况，具体取决于根是否有相等的秩。当两个秩不相等时，我们使具有高秩的根成为具有较低秩的根的父结点，但秩本身保持不变。当两个秩相同时，任选一个根作为父结点，并增加其秩的值。





## 例子



The result is much better than Straightforward method.





# 观察结论

令 $x$ 是任意节点， $p(x)$ 是 $x$ 的父节点，有下面两个基本的观察结论



## 观察结论 4.1

- $\text{rank}(p(x)) \geq \text{rank}(x) + 1.$



## 观察结论 4.2

- $\text{rank}(x)$ 的值初始化为0，在后继合并运算序列中递增，直到 $x$ 不再是根节点。一旦 $x$ 变成了其他节点的一个子节点，它的秩就不再改变了。



# 引理



引理 4.1 包括根节点x在内的树中节点的个数至少是 $2^{\text{rank}(x)}$ .

证明: 对合并运算的次数应用归纳法。

- (1) 最初, x自身形成一棵树, 它的秩为0;
- (2) 设x和y为两个根, 考虑运算UNION(x,y)。假定引理在这项运算之前成立;
- (3) 如果 $\text{rank}(x) < \text{rank}(y)$  (或 $\text{rank}(x) > \text{rank}(y)$ ), 以y (或x) 为根形成的树比老的以y (或x) 为根的树的节点多, 并且它的秩未改变。如果 $\text{rank}(x) \neq \text{rank}(y)$ , 引理4.1在运算之后成立。
- (4) 如果 $\text{rank}(x) = \text{rank}(y)$ , 根据归纳法, 在这种情况下, 以y为根形成的树至少有 $2^{\text{rank}(x)} + 2^{\text{rank}(y)} = 2^{\text{rank}(y)+1}$ 个节点。由于 $\text{rank}(y)$ 每次加1, 所以运算之后引理4.1成立。



# 算法分析

- 每次FIND运算的代价是 $O(\log n)$ .
  - 由引理4.1可知，在以x为根的树中的节点数是k，那么树的高度至多是 $\lfloor \log k \rfloor$ 。
- 运算UNION(x,y)所需要的时间：
  - 如果两个参数都是根的话，是 $O(1)$ 。如果x和y不都是根，那么运行时间减少到寻找运算的运行时间。因此，合并运算的时间复杂性和寻找运算的时间复杂性相同，都是 $O(\log n)$



# 算法分析



结论:

- 可以得出, 采用按秩合并措施,  $m$ 次合并和寻找指令的交替执行序列的时间复杂性是 $O(m \log n)$ 。

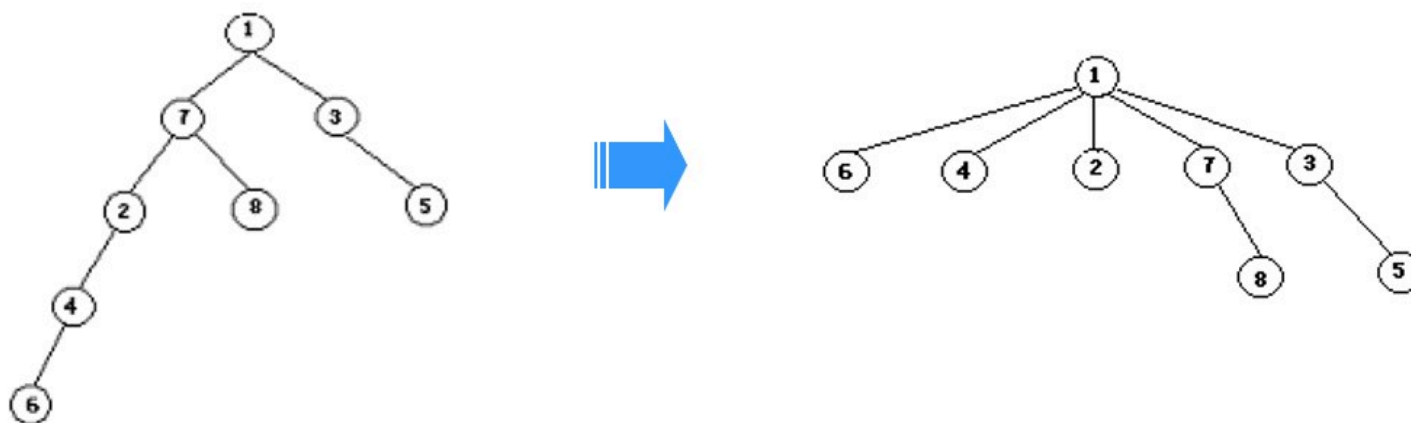


我们能否进一步加强**FIND**操作的效率?



## 边查询边“路径压缩”

其实，我们还能将集合查找的算法复杂度进一步降低：采用“路径压缩”算法。它的想法很简单：在集合的查找过程中顺便将树的深度降低。



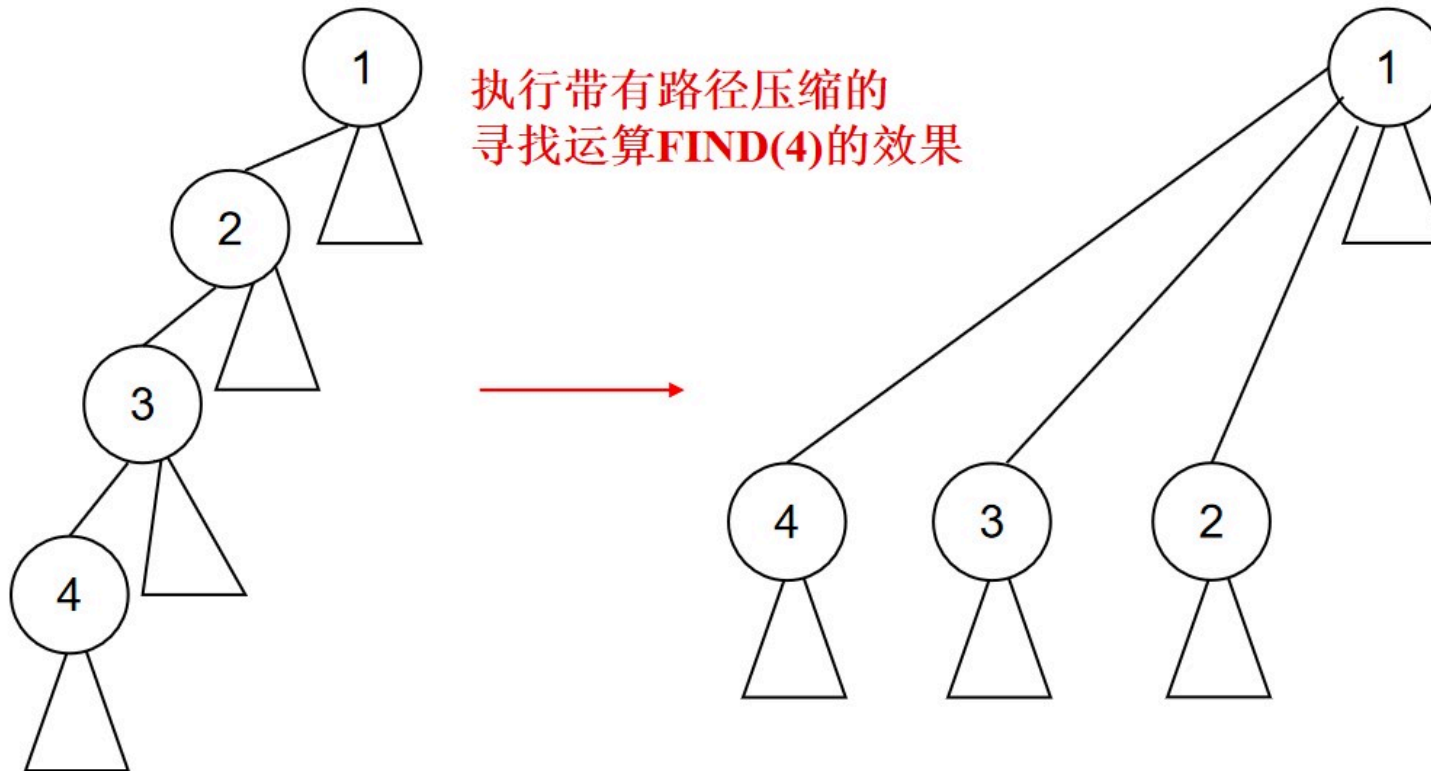


## 4.3.2 路径压缩

- 在 $\text{FIND}(x)$  运算中，找到根节点 $y$ 之后，我们再一次遍历从 $x$ 到 $y$ 的路径，并沿着路径改变所有节点指向父节点的指针，使它们直接指向 $y$ 。



## 例子





### 4.3.3 union-find 算法（路径压缩）

#### 算法 4.6 FIND

输入: 节点  $x$ .

输出:  $\text{root}(x)$ , 包含  $x$  的树的根。

1.  $y \leftarrow x$
2. **while**  $p(y) \neq \text{null}$       {寻找包含  $x$  的树的根}
3.      $y \leftarrow p(y)$
4. **end while**
5.  $\text{root} \leftarrow y$ ;     $y \leftarrow x$
6. **while**  $p(y) \neq \text{null}$       {执行路径压缩}
7.      $w \leftarrow p(y)$ ;     $p(y) \leftarrow \text{root}$ ;     $y \leftarrow w$
8. **end while**
9. **return**  $\text{root}$

华中农业大学信息学院





### 4.3.3 union-find算法（按秩合并）

#### 算法 4.7 UNION

**Input:** 两个元素 $x$ 和 $y$

**Output:** 包含 $x$ 和 $y$ 的两个树的合并，原来的树被破坏。

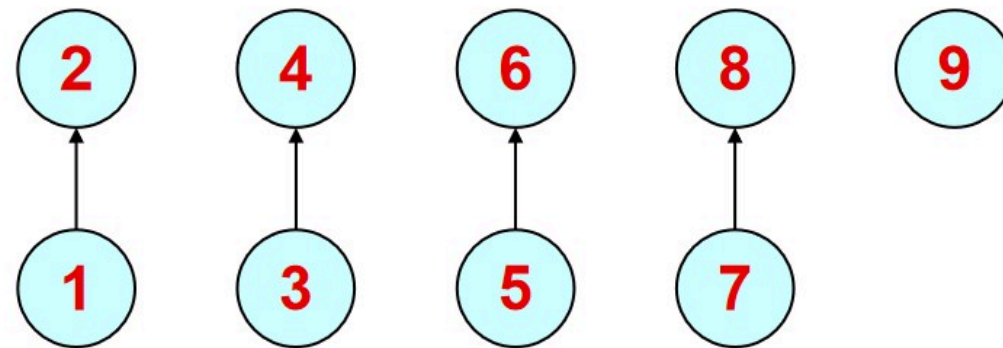
1.  $u \leftarrow \text{FIND}(x); v \leftarrow \text{FIND}(y)$
2. **if**  $\text{rank}(u) \leq \text{rank}(v)$  **then**
3.      $p(u) \leftarrow v$
4.     **if**  $\text{rank}(u) = \text{rank}(v)$  **then**  $\text{rank}(v) \leftarrow \text{rank}(v) + 1$
5. **else**  $p(v) \leftarrow u$
6. **end if**

华中农业大学信息学院



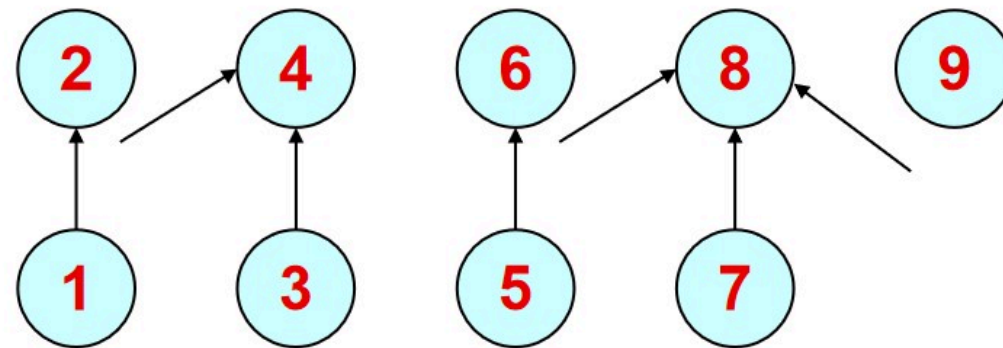
## 例子

- 设 $S=\{1,2,\dots,9\}$ ，考虑用下面的合并和寻找序列：  
UNION(1,2), UNION(3,4), UNION(5,6), UNION(7,8),  
UNION(2,4), UNION(8,9), UNION(6,8), FIND(5),  
UNION(4,8), FIND(1)。

 例子

$S \{ \text{UNION}(1,2), 9 \}$

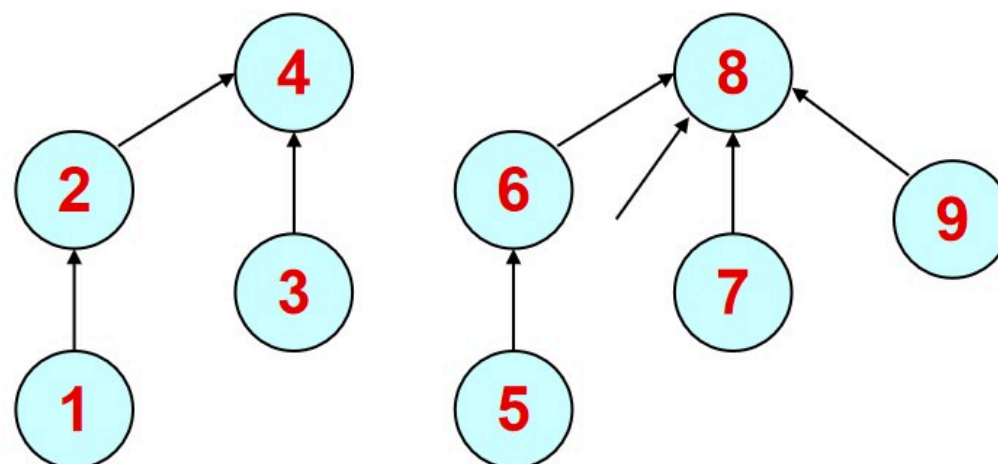
## 例子



UNION(8,9)

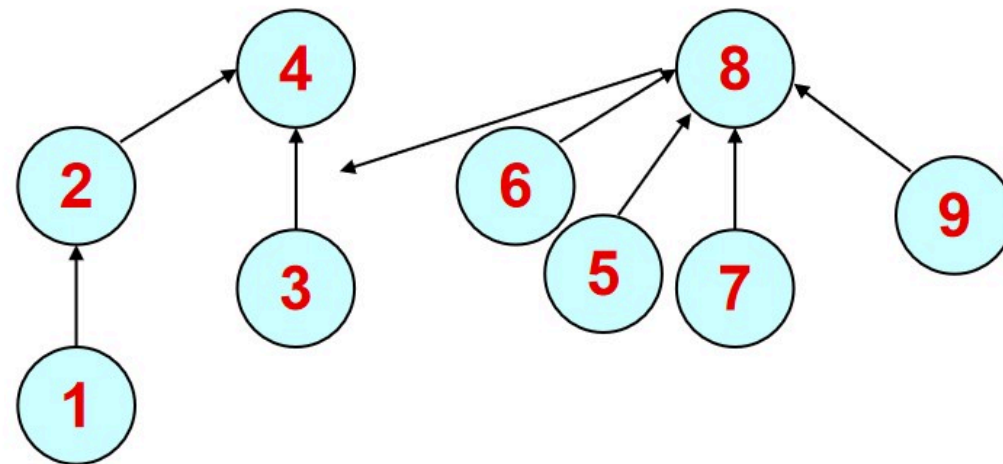


## 例子



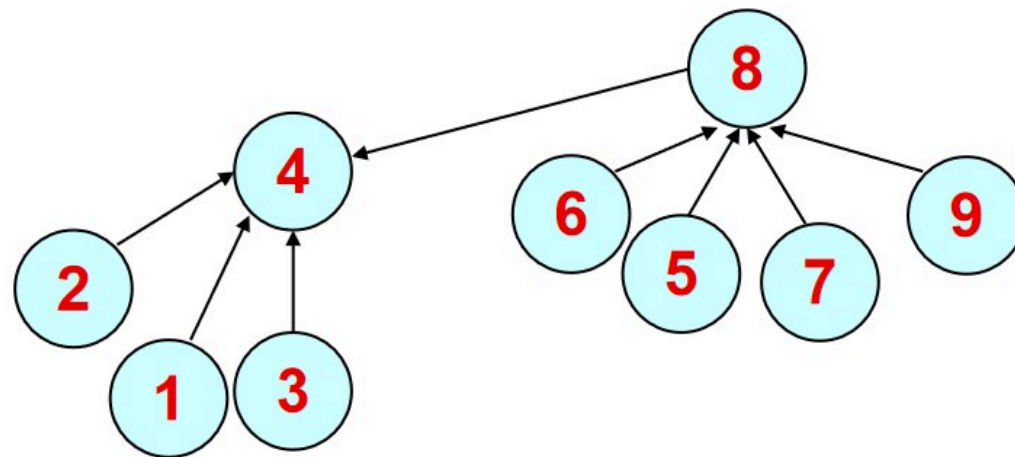
FIND(5)

## 例子



UNION(4,8)

## 例子



FIND(1)

End



## 4.3.4 union-find算法的分析

 引理 4.2 对于任意的整数:  $r \geq 0$ , 秩 $r$ 的节点数至多是  $n/2^r$ .

- 证明: 固定 $r$ 的一个特定值.
- 当给一个节点 $x$ 指定秩为 $r$ 时, 对以 $x$ 为根的树中包含的所有节点都用 $x$ 标号。由引理4.1可知, 标号的节点个数至少是 $2^r$ .
- 如果树的根节点发生变化, 那么新树的根节点的秩至少是 $r+1$ , 这说明那些用 $x$ 标号过的节点将不再被标号了。
- 由于标号过的节点个数的最大值是 $n$ , 而且每个秩为 $r$ 的根至少有 $2^r$ 个节点, 那么至多有 $n/2^r$ 个节点秩为 $r$ 。





## 4.3.4 union-find算法的分析

⇒ 推论 4.1 任何节点的秩最大是  $\lfloor \log n \rfloor$ .

证明: 如果对于某个节点  $x$ , 有  $\text{rank}(x) = r \geq \lfloor \log n \rfloor + 1$ , 那么由引理 4.2 可知, 至多有  $n/2^{\lfloor \log n \rfloor + 1} < 1$  个节点的秩为  $r$ .

• 定义 4.2

• 对于任意正整数  $n$ ,  $\log^* n$  定义为

$$\log^* n = \begin{cases} 0 & \text{if } n = 0 \text{ or } 1 \\ \min \{i \geq 0 \mid \underbrace{\log \log \dots \log n}_{i \text{ times}} \leq 1\} & \text{if } n \geq 2 \end{cases}$$

例如,  $\log^* 2 = 1$ ,  $\log^* 4 = 2$ ,  $\log^* 16 = 3$ ,  $\log^* 65536 = 4$ ,  $\log^* 2^{65536} = 5$ .



# 算法分析



## 定理 4.3

- 设 $T(m)$ 表示用按秩合并和路径压缩处理 $m$ 个合并和寻找运算的交替序列 $\sigma$ 所需的运行时间，那么在最坏情况下 $T(m)=O(m\log^*n)$
- 注意对于几乎所有的实际用途， $\log^*n \leq 5$ . 这说明事实上对于所有的实际应用，运行时间是 $O(m)$ 。



## 练习

- 4.3
- 4.7
- 4.11
- 4.26

华中农业大学信息学院